Министерство науки и высшего образования Российской Федерации
ФГБОУ ВО «Ростовский государственный экономический университет (РИНХ)»

ОТЧЁТ

по лабораторной работе № 1.7

«Работа с данными в XML»

по дисциплине «Разработка и сопровождение программных систем»

Вариант 13

Выполнил: студент группы ИСТ-321
Хасаев Умахан

Ростов-на-Дону
2026

Репозиторий с исходным кодом: https://github.com/warhogz/software-development-and-maintenance

Цель работы

Научиться сохранять и загружать данные JavaFX-приложения в формате XML, использовать компонент FileChooser для выбора файла, создавать меню приложения и хранить путь к последнему открытому файлу в пользовательских настройках.

Задание

1. Хранение данных в XML.
2. Использование компонента JavaFX FileChooser.
3. Использование компонента JavaFX Menu.
4. Сохранение пути к последнему открытому файлу в пользовательских настройках.

Индивидуальное задание (вариант 13): предметная область «Области – Районы».

Ход работы

Работа выполнена на основе проекта лабораторной работы 1.6. Структура проекта приведена в листинге 1.

Листинг 1 – Структура проекта

```
lab1_7
├── pom.xml
└── src/main
    ├── java
    │   ├── module-info.java
    │   └── ru/lab/regionapp
    │       ├── MainApp.java
    │       ├── model
    │       │   ├── District.java
    │       │   ├── Region.java
    │       │   └── RegionDataWrapper.java
    │       └── view
    │           ├── DistrictEditDialogController.java
    │           ├── DistrictOverviewController.java
    │           └── RootLayoutController.java
    └── resources/ru/lab/regionapp
        ├── images
        └── view
            ├── DistrictEditDialog.fxml
            ├── DistrictOverview.fxml
```

```
├── RegionTheme.css
└── RootLayout.fxml
```

1. Хранение данных в XML. Для преобразования объектов в XML и обратно используется технология JAXB (Jakarta XML Binding). В файл pom.xml добавлены зависимости jakarta.xml.bind-api и jaxb-runtime (листинг 2), а в module-info.java – требование requires jakarta.xml.bind и открытие пакета model для JAXB.

Листинг 2 – Зависимости JAXB в pom.xml

```
<dependency>
    <groupId>jakarta.xml.bind</groupId>
    <artifactId>jakarta.xml.bind-api</artifactId>
    <version>4.0.2</version>
</dependency>
<dependency>
    <groupId>org.glassfish.jaxb</groupId>
    <artifactId>jaxb-runtime</artifactId>
    <version>4.0.5</version>
</dependency>
```

Классы-модели подготовлены для JAXB. Название области помечено аннотацией @XmlID и служит идентификатором, а район хранит ссылку на область с помощью аннотации @XmlIDREF. Поэтому в XML у района записывается только название области, а при загрузке эта связь восстанавливается. Для сохранения сразу двух списков создан класс-обёртка RegionDataWrapper с аннотациями @XmlRootElement, @XmlElementWrapper и @XmlElement (листинг 3).

Листинг 3 – Класс-обёртка RegionDataWrapper

```
package ru.lab.regionapp.model;

import java.util.ArrayList;
import java.util.List;

import jakarta.xml.bind.annotation.XmlElement;
import jakarta.xml.bind.annotation.XmlElementWrapper;
import jakarta.xml.bind.annotation.XmlRootElement;

/**
 * Вспомогательный класс для сохранения списков областей и районов в XML.
 */
@XmlRootElement(name = "data")
public class RegionDataWrapper {

    private List<Region> regions = new ArrayList<>();
    private List<District> districts = new ArrayList<>();
```

```
    @XmlElementWrapper(name = "regions")
    @XmlElement(name = "region")
    public List<Region> getRegions() {
        return regions;
    }

    public void setRegions(List<Region> regions) {
        this.regions = regions;
    }

    @XmlElementWrapper(name = "districts")
    @XmlElement(name = "district")
    public List<District> getDistricts() {
        return districts;
    }

    public void setDistricts(List<District> districts) {
        this.districts = districts;
    }
}
```

Сохранение и загрузка реализованы в классе MainApp (листинг 4). Метод saveDataToFile() выполняет маршализацию (Marshaller) данных в XML-файл, метод loadDataFromFile() – демаршализацию (Unmarshaller) и заменяет содержимое наблюдаемых списков. Пример получившегося XML-файла приведён в листинге 5.

Листинг 4 – Методы работы с XML и настройками в классе MainApp

```
    /**
     * Возвращает путь к последнему открытому файлу данных. Информация считывается
     * из пользовательских настроек Java (в Windows – из реестра).
     *
     * @return файл или null, если путь не сохранён
     */
    public File getDataFilePath() {
        Preferences prefs = Preferences.userNodeForPackage(MainApp.class);
        String filePath = prefs.get("filePath", null);
        return filePath != null ? new File(filePath) : null;
    }

    /**
     * Задаёт путь к текущему загруженному файлу. Путь сохраняется в настройках
     * и отображается в заголовке окна.
     *
     * @param file файл или null, чтобы удалить путь
     */
    public void setDataFilePath(File file) {
        Preferences prefs = Preferences.userNodeForPackage(MainApp.class);
        if (file != null) {
            prefs.put("filePath", file.getPath());
            primaryStage.setTitle("Области и районы - " + file.getName());
        } else {
            prefs.remove("filePath");
            primaryStage.setTitle("Области и районы");
        }
    }
```

```
    /**
     * Загружает данные из указанного файла. Текущие данные будут заменены.
     */
    public void loadDataFromFile(File file) {
        try {
            JAXBContext context = JAXBContext.newInstance(RegionDataWrapper.class);
            Unmarshaller um = context.createUnmarshaller();

            // Чтение XML из файла и демаршализация.
            RegionDataWrapper wrapper = (RegionDataWrapper) um.unmarshal(file);

            regionData.clear();
            regionData.addAll(wrapper.getRegions());
            districtData.clear();
            districtData.addAll(wrapper.getDistricts());

            // Сохраняем путь к файлу в настройках.
            setDataFilePath(file);
        } catch (Exception e) {
            showError("Не удалось загрузить данные", "Не удалось загрузить данные из
файла:\n" + file.getPath(), e);
        }
    }

    /**
     * Сохраняет текущие данные в указанный файл.
     */
    public void saveDataToFile(File file) {
        try {
            JAXBContext context = JAXBContext.newInstance(RegionDataWrapper.class);
            Marshaller m = context.createMarshaller();
            m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);

            // Оборачиваем наши данные.
            RegionDataWrapper wrapper = new RegionDataWrapper();
            wrapper.setRegions(regionData);
            wrapper.setDistricts(districtData);

            // Маршализация и сохранение XML в файл.
            m.marshal(wrapper, file);

            // Сохраняем путь к файлу в настройках.
            setDataFilePath(file);
        } catch (Exception e) {
            showError("Не удалось сохранить данные", "Не удалось сохранить данные в
файл:\n" + file.getPath(), e);
        }
    }
```

Листинг 5 – Фрагмент XML-файла с данными (data_regions_example.xml)

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<data>
    <districts>
        <district>
            <adminCenter>Арзамас</adminCenter>
            <area>1923.0</area>
            <head>Иванов И.И.</head>
            <name>Арзамасский район</name>
            <region>Нижегородская область</region>
        </district>
        <district>
            <adminCenter>Богородск</adminCenter>
            <area>1300.0</area>
            <head>Смирнов А.В.</head>
```

```
            <name>Богородский район</name>
            <region>Нижегородская область</region>
        </district>
        <district>
            <adminCenter>Кстово</adminCenter>
            <area>1203.0</area>
            <head>Кузнецов Д.С.</head>
            <name>Кстовский район</name>
    ...
```

2–3. Меню и FileChooser. Корневой макет RootLayout.fxml дополнен меню: «Файл» (Новый, Открыть…, Сохранить, Сохранить как…, Выход) и «Справка» (О программе). Пункты меню связаны с методами контроллера RootLayoutController через атрибут onAction. В методах handleOpen() и handleSaveAs() используется компонент FileChooser с фильтром файлов *.xml; при сохранении к имени файла добавляется расширение .xml, если оно не указано. Метод handleSave() сохраняет данные в текущий файл, а если файл ещё не выбран – вызывает «Сохранить как…» (листинг 6).

Листинг 6 – Класс RootLayoutController

```
package ru.lab.regionapp.view;

import java.io.File;

import javafx.fxml.FXML;
import javafx.scene.control.Alert;
import javafx.scene.control.Alert.AlertType;
import javafx.stage.FileChooser;
import ru.lab.regionapp.MainApp;

/**
 * Контроллер корневого макета. Предоставляет базовые меню навигации приложения.
 */
public class RootLayoutController {

    // Ссылка на главное приложение
    private MainApp mainApp;

    public void setMainApp(MainApp mainApp) {
        this.mainApp = mainApp;
    }

    /**
     * Создаёт пустую книгу данных.
     */
    @FXML
    private void handleNew() {
        mainApp.getDistrictData().clear();
        mainApp.getRegionData().clear();
        mainApp.setDataFilePath(null);
    }

    /**
     * Открывает FileChooser, чтобы пользователь смог выбрать XML-файл для загрузки.
```

```
     */
    @FXML
    private void handleOpen() {
        FileChooser fileChooser = new FileChooser();
        fileChooser.setTitle("Открыть файл данных");
        fileChooser.getExtensionFilters().add(new FileChooser.ExtensionFilter("XML
файлы (*.xml)", "*.xml"));

        File file = fileChooser.showOpenDialog(mainApp.getPrimaryStage());
        if (file != null) {
            mainApp.loadDataFromFile(file);
        }
    }

    /**
     * Сохраняет файл в открытый в настоящее время файл. Если файл не открыт,
     * то отображается диалог «Сохранить как».
     */
    @FXML
    private void handleSave() {
        File file = mainApp.getDataFilePath();
        if (file != null) {
            mainApp.saveDataToFile(file);
        } else {
            handleSaveAs();
        }
    }

    /**
     * Открывает FileChooser, чтобы пользователь смог выбрать файл для сохранения.
     */
    @FXML
    private void handleSaveAs() {
        FileChooser fileChooser = new FileChooser();
        fileChooser.setTitle("Сохранить файл данных");
        fileChooser.getExtensionFilters().add(new FileChooser.ExtensionFilter("XML
файлы (*.xml)", "*.xml"));

        File file = fileChooser.showSaveDialog(mainApp.getPrimaryStage());
        if (file != null) {
            // Проверяем, что файл имеет правильное расширение.
            if (!file.getPath().endsWith(".xml")) {
                file = new File(file.getPath() + ".xml");
            }
            mainApp.saveDataToFile(file);
        }
    }

    /**
     * Открывает диалог «О программе».
     */
    @FXML
    private void handleAbout() {
        Alert alert = new Alert(AlertType.INFORMATION);
        alert.initOwner(mainApp.getPrimaryStage());
        alert.setTitle("Области и районы");
        alert.setHeaderText("О программе");
        alert.setContentText("Автор: Хасаев Умахан, группа ИСТ-331\nДанные хранятся в
XML-файле.");
        alert.showAndWait();
    }

    /**
     * Закрывает приложение.
     */
    @FXML
    private void handleExit() {
```

```
            System.exit(0);
        }
    }
```

4. Сохранение пути в настройках. Путь к последнему открытому файлу сохраняется с помощью класса java.util.prefs.Preferences методом prefs.put("filePath", ...) в методе setDataFilePath(), а читается методом getDataFilePath(). В Windows настройки хранятся в реестре. При запуске приложения в методе start() проверяется, сохранён ли путь, и если файл существует, данные загружаются из него автоматически. Имя файла выводится в заголовке окна.

Результаты работы программы

Результат работы приложения показан на рисунках 1–8.

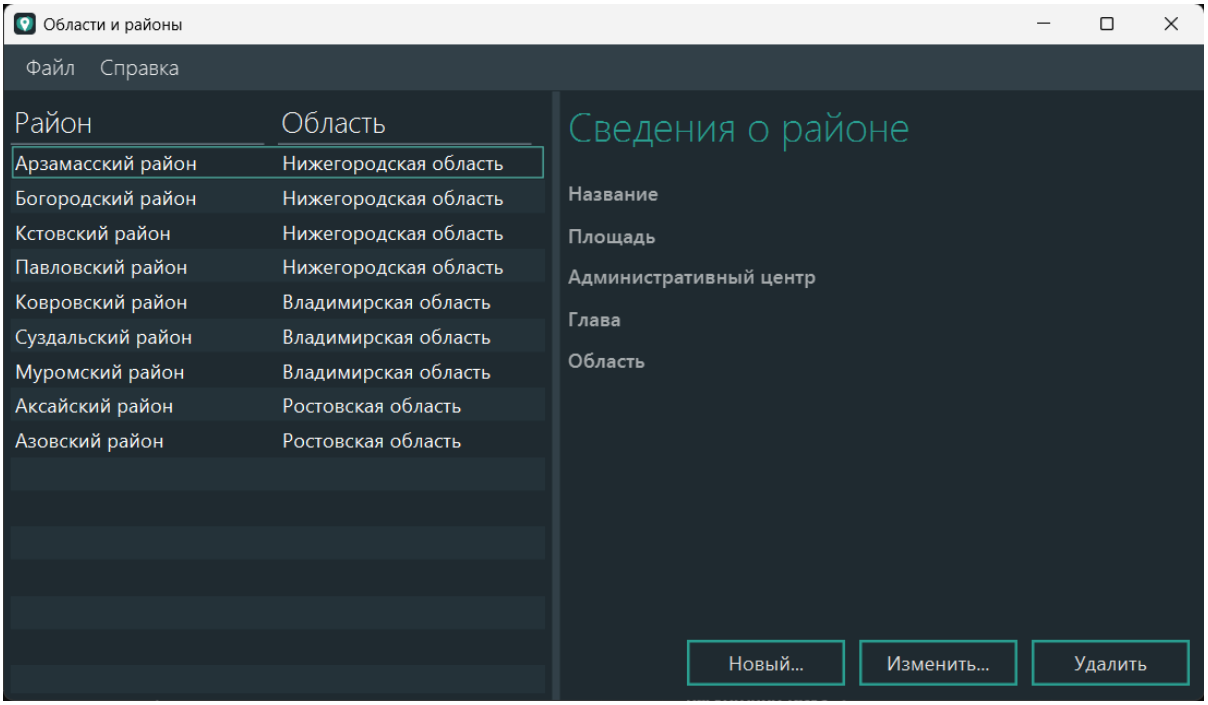


Рисунок 1 – Главное окно приложения при первом запуске

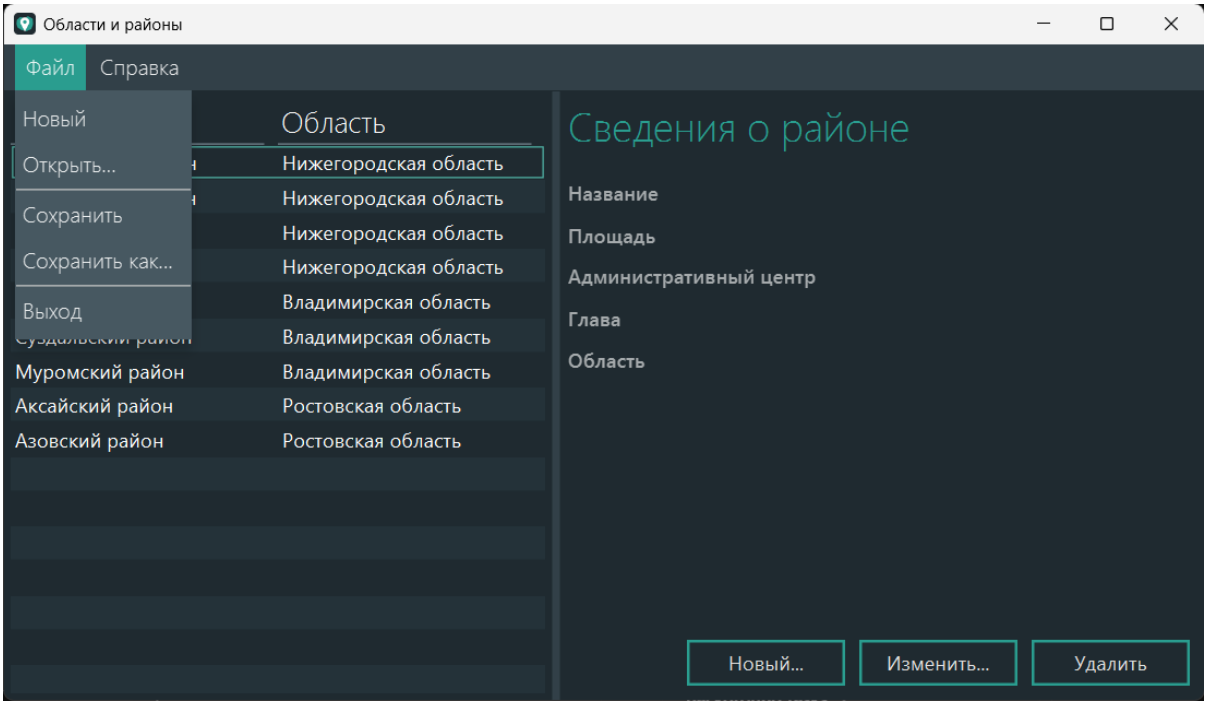


Рисунок 2 – Меню «Файл»

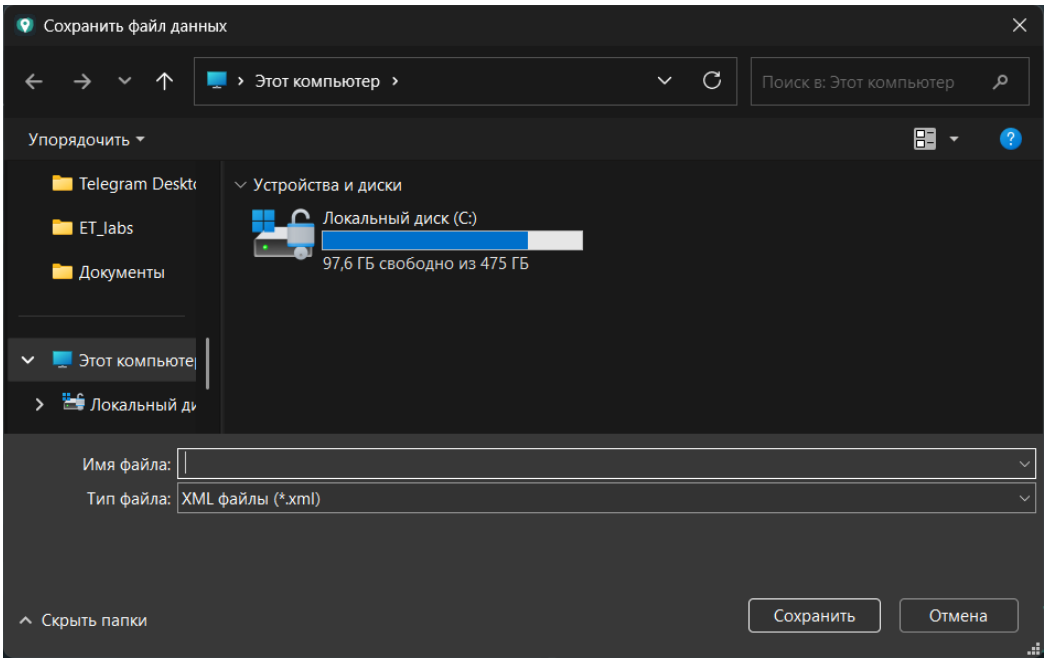


Рисунок 3 – Окно выбора файла для сохранения (FileChooser)

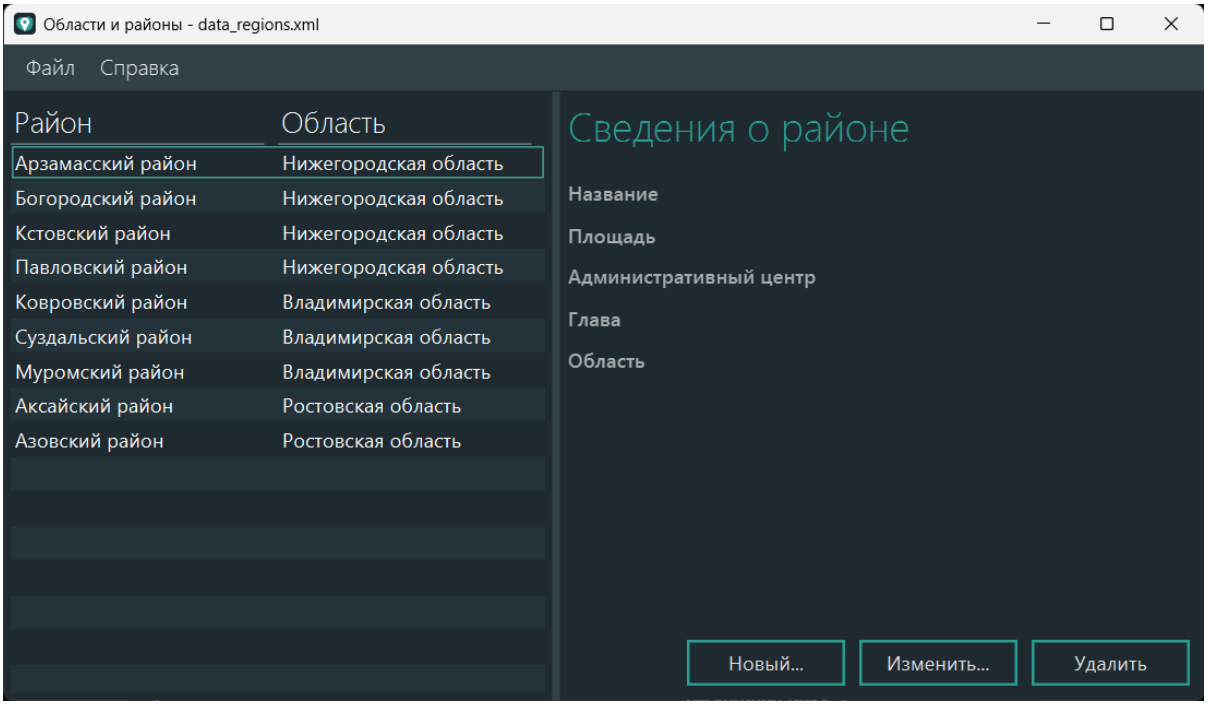


Рисунок 4 – После сохранения имя файла отображается в заголовке окна

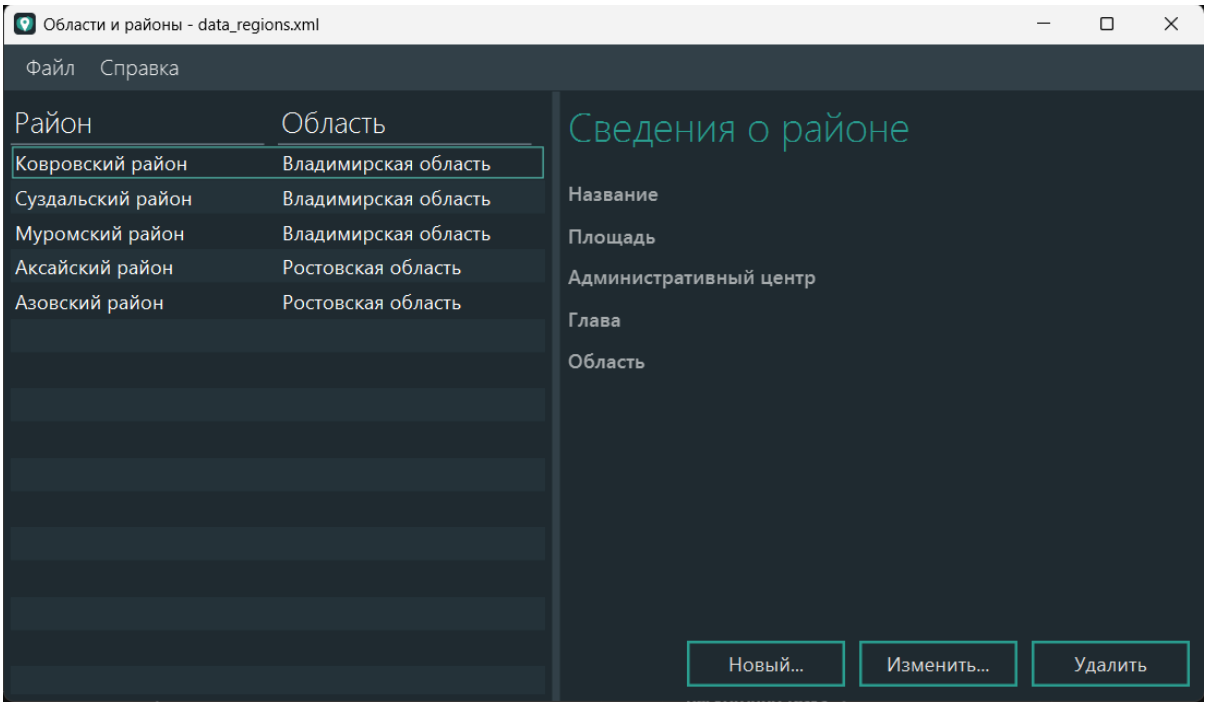


Рисунок 5 – Данные изменены: часть районов удалена

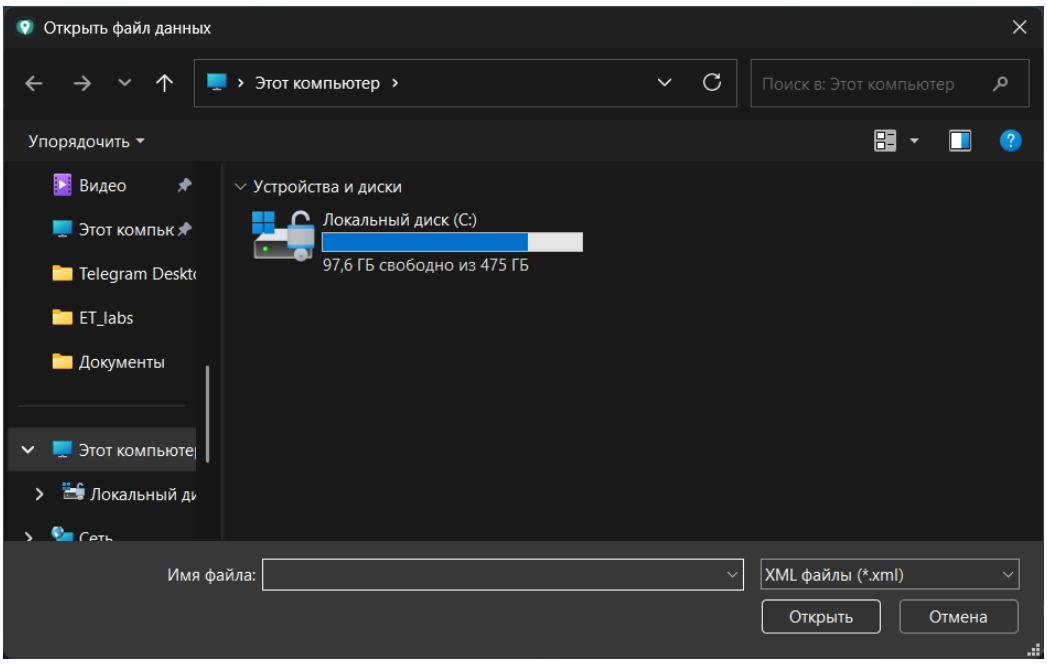


Рисунок 6 – Окно выбора файла для открытия (FileChooser)

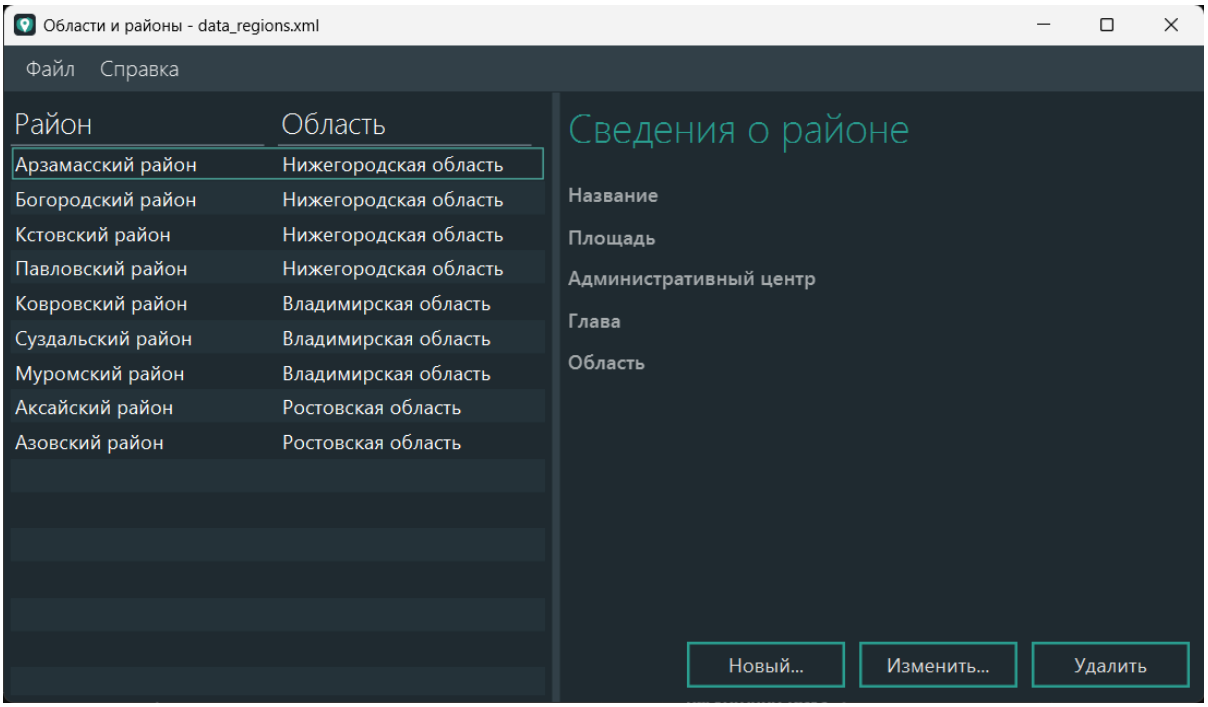


Рисунок 7 – Данные загружены из XML-файла

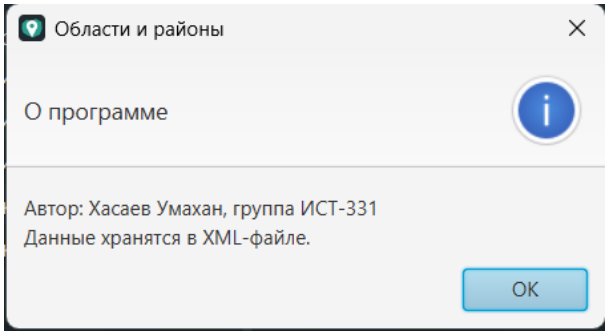


Рисунок 8 – Окно «О программе»

Вывод

В ходе выполнения лабораторной работы приложение «Области и районы» было дополнено сохранением и загрузкой данных в формате XML с помощью JAXB, меню с командами работы с файлами и компонентом FileChooser для выбора файла. Путь к последнему открытому файлу хранится в пользовательских настройках (Preferences), поэтому при следующем запуске данные загружаются автоматически.